

SỔ TAY THỰC HÀNH APRILTAG VỚI JETBOT

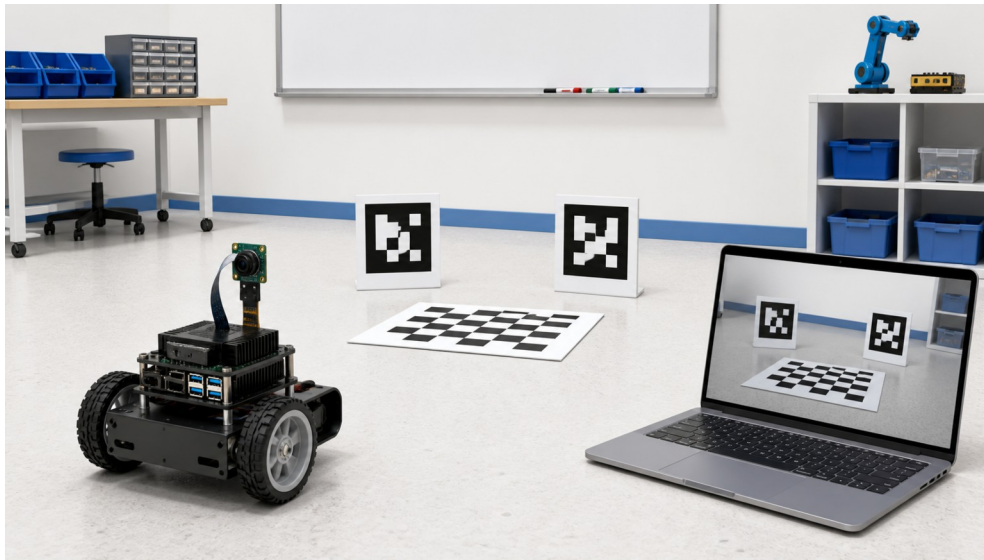
Từ camera - hiệu chỉnh - nhận diện đến robot tự điều hướng

THỂ TRIỂN KHAI NHANH CHO GIÁO VIÊN

Dùng trang này trước giờ học để xác nhận thiết bị, chọn đúng notebook và bảo đảm robot chỉ chạy trong điều kiện an toàn.

Mốc kiểm tra	Điều giáo viên cần xác nhận
Trước khi học sinh vào lớp	Mở JupyterLab, thử camera live, kiểm tra pin/camera CSI/Wi-Fi, chuẩn bị tag tag36h11 và checkerboard đúng mẫu.
Trước khi robot chạy	Bài 5 phải đọc ID và z_m ổn định; dọn vùng trống tối thiểu 1,5 m × 1,5 m; có người quan sát và sẵn cell/lệnh robot.stop().
Khi có lỗi	Dừng robot → Interrupt Kernel → chạy robot.stop() → kiểm tra notebook gốc, calibration, TAG_SIZE và ánh sáng; không tự đoán tham số.
Sản phẩm tối thiểu	Ảnh camera live, camera_calibration.json + RMS, ảnh/ghi chú verify, bảng đo 40/60/80 cm, và minh chứng chạy đến tag.

Nguyên tắc cốt lõi: không tự ghép mã minh hoạ thành chương trình điều khiển mới; chỉ chạy notebook gốc đã được kiểm thử trên đúng JetBot của lớp.



Hình 1. Mô hình thực hành: JetBot, camera, AprilTag, checkerboard và máy tính.

Dành cho	Kết quả cuối
Học sinh mới bắt đầu	JetBot nhận diện tag, ước lượng khoảng cách, tự tìm tag và dừng an toàn.

Cách học bằng sổ tay này

- Mỗi bài: chuẩn bị - đọc giải thích - làm quy trình - chạy mã - lưu sản phẩm.
- Không chạy cả notebook cùng lúc; sau mỗi cell phải có một kết quả quan sát được.
- Khi xe chạy, luôn có người quan sát cạnh xe và vùng trống tối thiểu 1.5 m x 1.5 m.
- Không xóa camera_calibration.json: đó là “thước đo” của camera cho các bài sau.

Bản đồ dự án

Bài	Robot học được gì	Sản phẩm
1	Hiểu AprilTag	Bảng quan sát
2	Nhìn qua camera	Ảnh camera live
3	Đo đúng hơn	camera_calibration.json
4	Kiểm tra thước đo	Ảnh khử méo
5	Đọc tag	Bảng ID/z_m
6	Tới một tag	Ảnh/video xe dừng
7	Tới hai tag	Bảng tính chính

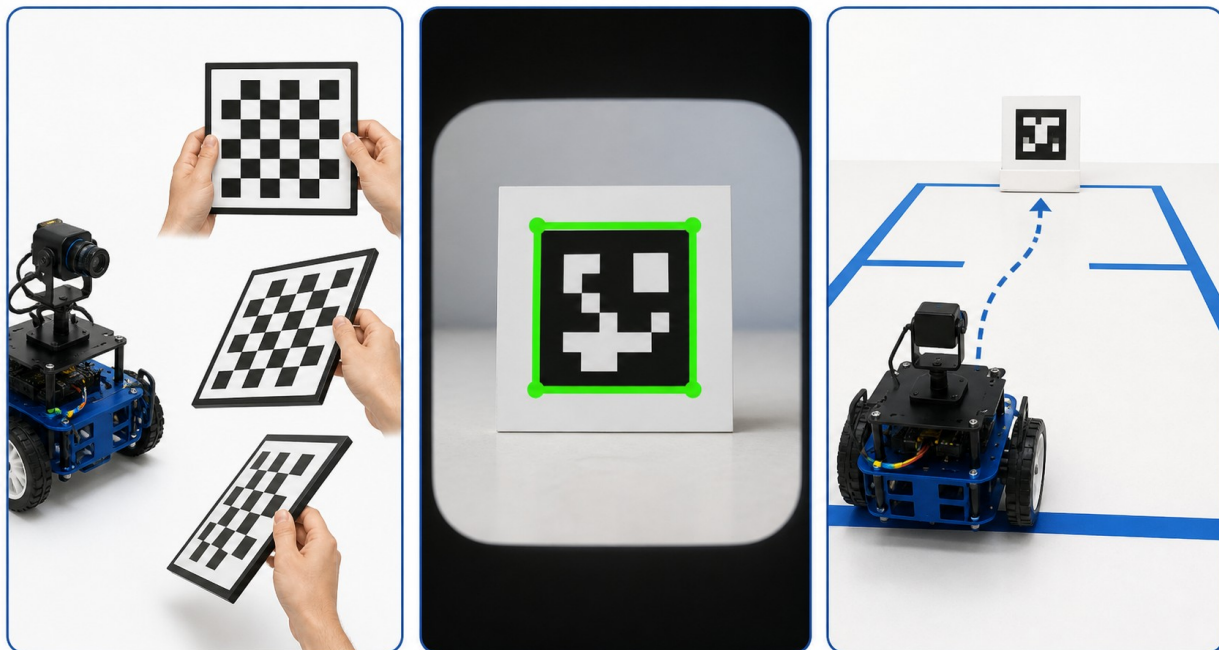
BẮT ĐẦU TỪ ĐÂU? - Quy trình tổng cho giáo viên và học sinh

Không nhảy thẳng đến bài xe chạy. Chỉ chuyển sang bước sau khi có sản phẩm đầu ra của bước trước.

Thứ tự	Việc phải làm	Điều kiện chuyển tiếp
0	Giáo viên chuẩn bị thiết bị, Wi-Fi, tag và checkerboard	JetBot bật được, camera gắn chắc, có IP.
1	Bài 1: hiểu tag	Học sinh biết ID, góc, tâm và khoảng cách.
2	Bài 2: test camera	Có ảnh live trong JupyterLab.
3	Bài 3: calibration	Đã lưu camera_calibration.json, RMS hợp lý.
4	Bài 4: verify	Xác nhận ảnh khử méo và đọc được RMS.
5	Bài 5: perception	Đọc ổn định ID và z_m của một tag.
6	Bài 6: navigation 1 tag	Robot dừng an toàn gần mục tiêu.
7	Bài 7: multi-tag	Có dữ liệu tinh chỉnh và robot đến hai tag.

Luồng quyết định khi có lỗi

Camera không lên -> tắt kernel cũ/reboot. Camera lên nhưng không thấy tag -> kiểm tra family, ánh sáng, khoảng cách và tag. Thấy tag nhưng z_m sai -> kiểm tra TAG_SIZE và calibration. z_m đúng nhưng xe chạy sai -> giảm tốc độ, kiểm tra x_m và tolerance.



Hình 2. Ba thao tác cốt lõi: hiệu chỉnh camera, nhận diện tag và điều hướng robot.

BÀI 1. AprilTag là gì?

Mục	Nội dung
Tài liệu nền	Tiết 24 - Robotic Laser Coating Removal và AprilTag
Mục tiêu	Giải thích được AprilTag là mốc thị giác để robot định vị tương đối trong phòng.
Sản phẩm	Bảng quan sát 4 thử nghiệm.

Cần những gì

- In hoặc mở AprilTag ID 0 và ID 1.
- Giấy ghi chép; bài này chưa cần JetBot.

Vật tư giáo viên và chuẩn bị trước giờ

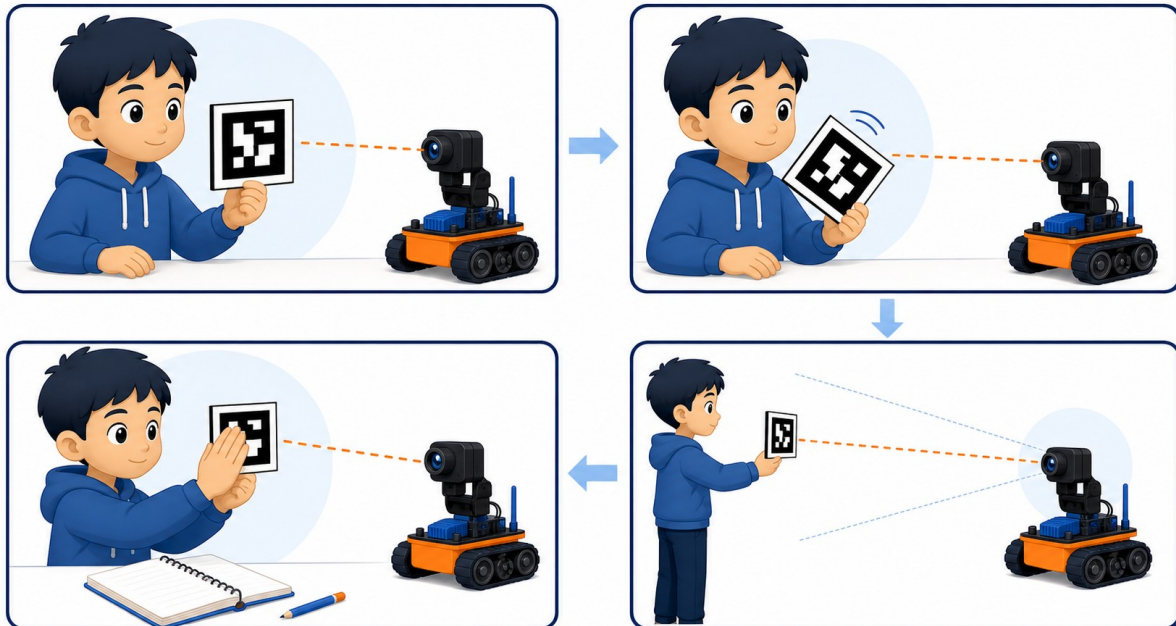
- In AprilTag ID 0-3 (tag36h11), dán bìa cứng; 1 bộ/nhóm.
- Máy chiếu hoặc ảnh/video minh họa bối cảnh robot định vị trong nhà.

Giải thích dễ hiểu

GPS trong phòng thường không chính xác đủ để robot di chuyển. AprilTag là hình vuông đen trắng có ID riêng. Camera tìm viền, bốn góc và mẫu ở giữa để biết tag là gì, nằm lệch bên nào và xa gần ra sao. QR code ưu tiên chứa nội dung; AprilTag ưu tiên hỗ trợ robot đo vị trí.

Quy trình làm

Sơ đồ trực quan: quan sát tag → xoay → thay đổi khoảng cách → che một phần → ghi nhận kết quả.



1. Quan sát viền đen, nền trắng và mẫu mã ở giữa.
2. Xoay tag 45 độ, đưa tag xa dần.
3. Che 1/4, 1/2 rồi 3/4 tag; dự đoán khả năng nhận diện.

4. Viết 3 câu: tag làm gì, vì sao không dùng GPS, tag khác QR code thế nào.

Mã mẫu

Chạy theo từng cell. Nếu notebook của lớp có tên hàm khác, giữ nguyên ý tưởng và thay bằng tên đúng trong notebook.

```
# Không cần code.
# Bảng ghi chép: điều kiện | dự đoán | kết quả
# thẳng | nghiêng | xa/gần | che 1/4 | che 1/2
```

Tự kiểm tra

- Nói được ID, center, corners và z_m nghĩa là gì.
- Biết hamming thấp và margin cao thường đáng tin hơn.

Lỗi hay gặp

Hiện tượng	Vì sao xảy ra?	Cách xử lý theo thứ tự
Chưa hiểu z_m	Chưa hình dung trực khoảng cách từ camera đến tag.	Đặt camera nhìn thẳng vào tag; xem z_m là khoảng cách tiến thẳng từ camera đến tag.
Nhầm QR code	Hai mã đều đen trắng nên dễ nhầm mục đích sử dụng.	Nhắc lại: QR ưu tiên chứa nội dung; AprilTag ưu tiên giúp robot định vị.

Xem thêm

- AprilTag official: <https://april.eecs.umich.edu/software/apriltag.html>

BÀI 2. Mở camera JetBot

Mục	Nội dung
Tài liệu nền	Tiết 25 - Cấu hình môi trường & Test camera
Mục tiêu	Thấy video trực tiếp từ camera CSI trong JupyterLab.
Sản phẩm	Ảnh chụp màn hình video và IP JetBot.

Cần những gì

- JetBot đã reboot.
- Máy tính cùng Wi-Fi, biết IP JetBot.
- Camera CSI gắn chặt và không bị che.

Vật tư giáo viên và chuẩn bị trước giờ

- JetBot đã reboot trước giờ; danh sách IP từng JetBot.
- Wi-Fi ổn định; giáo viên mở sẵn một notebook camera mẫu.

Giải thích dễ hiểu

Camera là “mắt” của JetBot. Một camera chỉ nên được một notebook dùng tại một thời điểm. Vì vậy trước khi mở notebook, hãy tắt tất cả kernel cũ để không bị lỗi camera bạn.

Quy trình làm

Sơ đồ trực quan: kết nối → đóng kernel cũ → kiểm tra camera → xem ảnh live → dừng an toàn.



5. Mở `http://<IP_JetBot>:8888`.
6. Chọn Kernel > Shut Down All Kernels.
7. Chạy cell import, reset camera và tạo camera.
8. Quan sát video ổn định 2 phút.

9. Kết thúc bằng camera.stop().

Mã mẫu

Chạy theo từng cell. Nếu notebook của lớp có tên hàm khác, giữ nguyên ý tưởng và thay bằng tên đúng trong notebook.

```
import cv2
from jetbot import Camera
if Camera._instance is not None:
    Camera._instance.stop(); Camera._instance=None
camera=Camera.instance(width=640,height=480,fps=10)
print(camera.value.shape) # phải thấy ảnh dạng mảng
```

Tự kiểm tra

- Ảnh live không bị đứng.
- Biết Shut Down All Kernels trước notebook khác.

Lỗi hay gặp

Hiện tượng	Vì sao xảy ra?	Cách xử lý theo thứ tự
Device busy	Một kernel/notebook cũ vẫn đang chiếm camera.	Chọn Shut Down All Kernels, chạy lại từ cell đầu; nếu chưa được thì reboot JetBot.
Không có hình	Dây CSI lỏng, sai IP/Wi-Fi hoặc camera bị che.	Kiểm tra dây camera, IP/Wi-Fi, tháo vật che; reboot nếu cần.
Lỗi khởi tạo	JetBot chưa boot xong hoặc camera chưa được giải phóng.	Reboot, chờ boot hoàn tất, rồi mở lại JupyterLab và chạy từng cell.

Xem thêm

- JetBot documentation: <https://jetbot.org/master/>

BÀI 3. Hiệu chỉnh camera

Mục	Nội dung
Tài liệu nền	Tiết 26 - Camera calibration
Mục tiêu	Tạo camera_calibration.json, ưu tiên RMS dưới 1.0 px.
Sản phẩm	camera_calibration.json và bảng 10-15 góc chụp.

Cần những gì

- Camera chạy được ở Bài 2.
- Checkerboard 11 x 8 ô, dán phẳng.
- Phòng đủ sáng, không bóng che bảng.

Vật tư giáo viên và chuẩn bị trước giờ

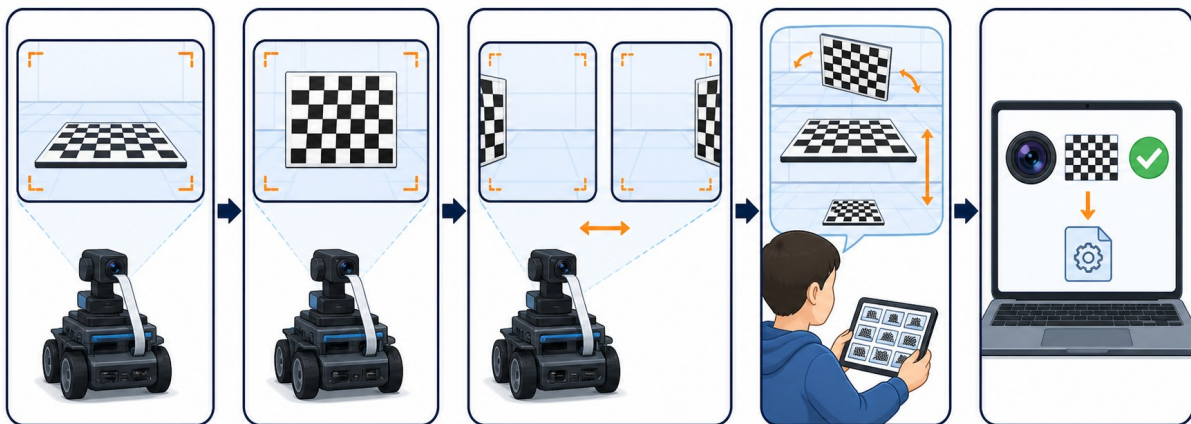
- Checkerboard 11 x 8 ô đã in phẳng; tối thiểu 1 bảng/nhóm.
- Kiểm tra ánh sáng, pin JetBot và dung lượng lưu tệp trước giờ.

Giải thích dễ hiểu

Ống kính làm méo ảnh ở rìa. Calibration tìm ra đặc trưng camera để khoảng cách AprilTag đáng tin hơn. Với bảng 11 x 8 ô, số góc bên trong là (10, 7), không phải (11, 8).

Quy trình làm

Sơ đồ trực quan: đặt checkerboard ở giữa, mép, nghiêng, gần/xạ → lưu kết quả hiệu chỉnh.



10. Mở notebook calibration, Shut Down All Kernels.
11. Đặt CHECKERBOARD = (10, 7).
12. Chỉ chụp khi phần mềm nhìn thấy đủ góc bảng.
13. Chụp 10-15 frame: giữa, mép, nghiêng, gần, xa.
14. Finish calibration, lưu RMS và giữ tệp JSON.

Mã mẫu

Chạy theo từng cell. Nếu notebook của lớp có tên hàm khác, giữ nguyên ý tưởng và thay bằng tên đúng trong notebook.

```
import json
CHECKERBOARD=(10,7)
with open("camera_calibration.json") as f:
    c=json.load(f)
print("RMS",c["rms_error"])
print("fx fy",c["fx"],c["fy"])
```

Tự kiểm tra

- RMS <1.0 px: tốt; 1.0–2.0 px: chấp nhận được sau khi kiểm tra ảnh khử méo; >2.0 px: nên hiệu chỉnh lại.
- Hiểu vì sao ảnh chụp cùng một góc là không đủ.

Lỗi hay gặp

Hiện tượng	Vì sao xảy ra?	Cách xử lý theo thứ tự
Không thấy checkerboard	Bảng cong, thiếu sáng hoặc chưa nằm trọn trong khung hình.	Làm phẳng bảng, tăng ánh sáng và đưa toàn bộ bảng vào ảnh.
RMS cao	Các ảnh chụp lặp góc hoặc bảng bị mờ/rung.	Reset và chụp 10–15 góc thật khác nhau: giữa, mép, nghiêng, gần, xa.
Sai kích thước	Đếm số ô thay vì số góc trong.	Với bảng 11 × 8 ô, đặt số góc trong là (10, 7).

Xem thêm

- OpenCV camera calibration: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

BÀI 4. Kiểm tra calibration

Mục	Nội dung
Tài liệu nền	Tiết 27 - Kiểm tra & phân tích kết quả hiệu chỉnh
Mục tiêu	Xác nhận “thước đo” camera ổn trước khi đo tag.
Sản phẩm	verify_undistorted.jpg và ghi chú RMS/fx/fy/cx/cy.

Cần những gì

- camera_calibration.json.
- JetBot/camera hoạt động.
- Checkerboard dự phòng.

Vật tư giáo viên và chuẩn bị trước giờ

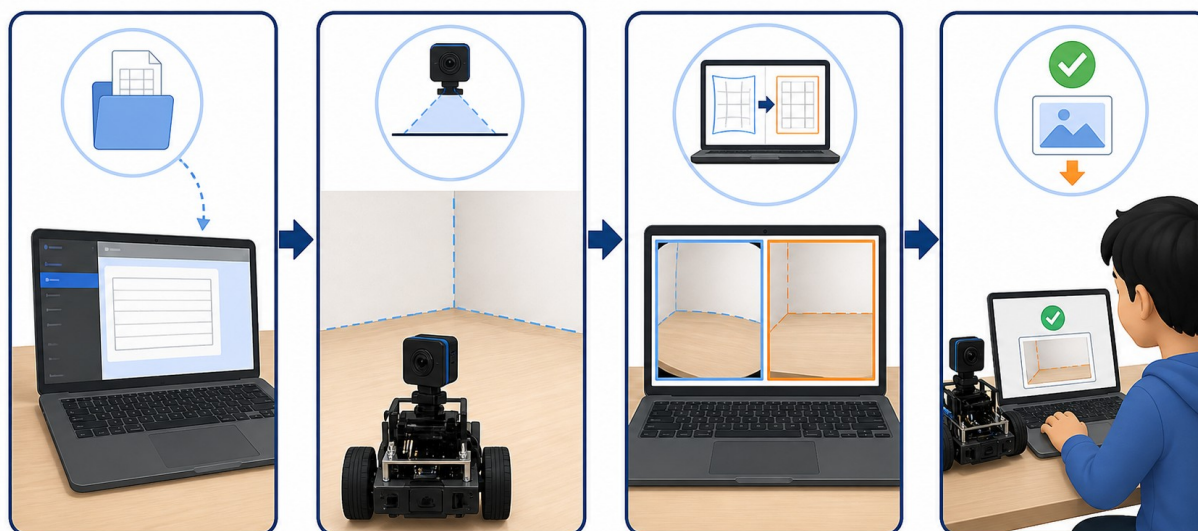
- Kiểm tra camera_calibration.json của từng nhóm từ tiết trước.
- Chuẩn bị checkerboard dự phòng để nhóm RMS cao làm lại.

Giải thích dễ hiểu

JSON lưu fx/fy (đặc trưng tiêu cự theo pixel), cx/cy (tâm quang học), dist_coeffs (mức méo) và RMS. Nếu bài sau đo z_m sai, đây là nơi cần kiểm tra trước khi chỉnh tốc độ xe.

Quy trình làm

Sơ đồ trực quan: mở file hiệu chỉnh → quan sát đường thẳng → so sánh trước/sau khử méo → lưu ảnh kiểm tra.



15. Mở JSON, ghi RMS.
16. Chạy verify để xem ảnh đã khử méo.
17. So sánh đường thẳng ở mép ảnh trước/sau.
18. Nếu RMS > 2.0, quay lại Bài 3.

19. Cleanup camera.

Mã mẫu

Chạy theo từng cell. Nếu notebook của lớp có tên hàm khác, giữ nguyên ý tưởng và thay bằng tên đúng trong notebook.

```
import cv2,json,numpy as np
with open("camera_calibration.json") as f:c=json.load(f)
K=np.array([[c["fx"],0,c["cx"]],[0,c["fy"],c["cy"]],[0,0,1]])
fixed=cv2.undistort(camera.value,K,np.array(c["dist_coeffs"]))
cv2.imwrite("verify_undistorted.jpg",fixed)
```

Tự kiểm tra

- Nói được vì sao RMS nhỏ là tốt.
- Biết không xóa JSON.

Lỗi hay gặp

Hiện tượng	Vì sao xảy ra?	Cách xử lý theo thứ tự
Không có JSON	File chưa được lưu hoặc đang mở sai thư mục notebook.	Tìm đúng thư mục; nếu không có file thì quay lại Bài 3 để hiệu chỉnh lại.
Verify không chạy	Camera còn bị kernel cũ giữ hoặc chưa được khởi tạo lại.	Tắt kernel cũ, tạo lại camera rồi chạy cell verify từ đầu.
RMS cao	Kết quả calibration ban đầu chưa đủ tốt.	Quay lại Bài 3, chụp lại bộ ảnh đa dạng hơn rồi tạo JSON mới.

Xem thêm

- OpenCV undistort reference: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html

BÀI 5. Nhận diện AprilTag

Mục	Nội dung
Tài liệu nền	Tiết 28 - Nhận thức về AprilTag
Mục tiêu	Đọc ID, vị trí, độ tin cậy và khoảng cách từ tag.
Sản phẩm	Bảng thước đo/z_m và ảnh tag có khung bao.

Cần những gì

- camera_calibration.json.
- Tag family tag36h11, đo cạnh thật bằng thước.
- TAG_SIZE đúng theo mét; 15 cm = 0.15.

Vật tư giáo viên và chuẩn bị trước giờ

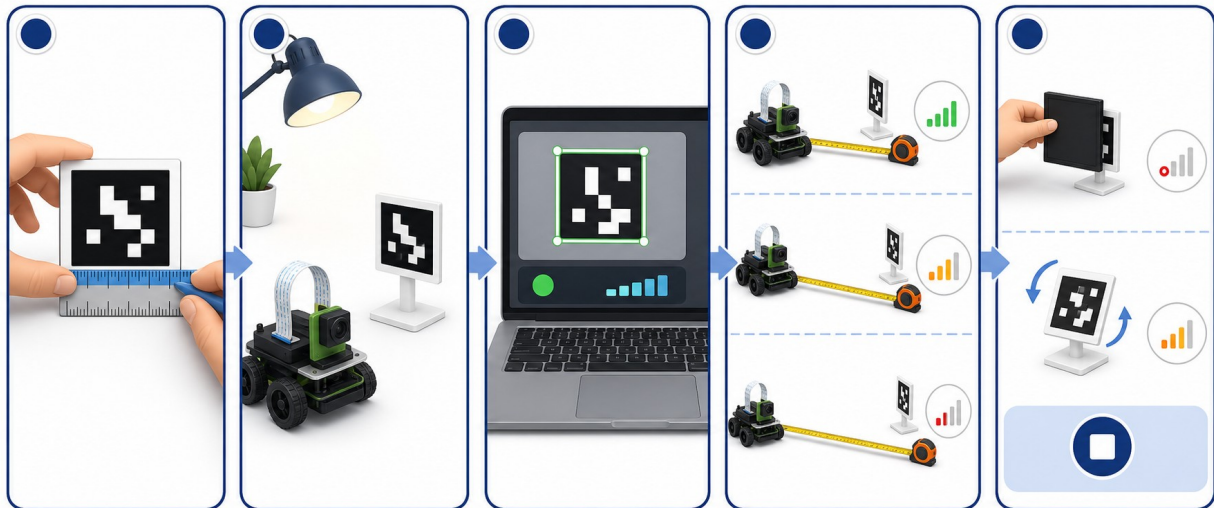
- Tag tag36h11 ID 0-3, thước đo và khu vực ánh sáng ổn định.
- Một file notebook Perception đã cài thư viện apriltag.

Giải thích dễ hiểu

Detector đọc ID và bốn góc. z_m là khoảng cách ước lượng; x_m/y_m là độ lệch theo trái-phải/lên-xuống. Tag nghiêng nhiều làm z_m kém chính xác; TAG_SIZE sai làm mọi khoảng cách sai dù ID vẫn đúng.

Quy trình làm

Sơ đồ trực quan: đo cạnh tag → đặt tag đủ sáng → nhận diện → đo ở nhiều khoảng cách → thử nghiêng/che tag.



20. Mở Perception sau Shut Down All Kernels.
21. Đưa một tag thẳng, đủ sáng, cách 40-70 cm.
22. Đo thật ở 40,60,80 cm rồi ghi z_m.
23. Che 1/4, 1/2, 3/4 và nghiêng 30, 45, 60 độ.
24. Cleanup.

Mã mẫu

Chạy theo từng cell. Nếu notebook của lớp có tên hàm khác, giữ nguyên ý tưởng và thay bằng tên đúng trong notebook.

```
import cv2, apriltag, numpy as np, json
with open("camera_calibration.json") as f: c=json.load(f)
fx,fy,cx,cy=c["fx"],c["fy"],c["cx"],c["cy"]; TAG_SIZE=.15
detector=apriltag.Detector(apriltag.DetectorOptions(families="tag36h11"))
gray=cv2.cvtColor(camera.value,cv2.COLOR_BGR2GRAY)
for tag in detector.detect(gray):
    w=np.linalg.norm(tag.corners[0]-tag.corners[1]); z=fx*TAG_SIZE/w
    x=(tag.center[0]-cx)*z/fx
    print(tag.tag_id,round(x,2),round(z,2),tag.hamming,tag.decision_margin)
```

Tự kiểm tra

- Hamming=0 là lý tưởng; margin cao thường rõ hơn.
- Biết x_m và z_m dùng cho navigation.

Lỗi hay gặp

Hiện tượng	Vì sao xảy ra?	Cách xử lý theo thứ tự
Không detect	Sai family/ID, thiếu sáng, tag quá xa hoặc không nằm trọn trong ảnh.	Dùng tag36h11 đúng ID, tăng sáng, đưa tag gần hơn và đặt trọn tag trong khung.
z_m sai	TAG_SIZE sai hoặc dùng camera_calibration.json không đúng camera.	Đo lại cạnh tag theo mét; kiểm tra và dùng đúng file calibration của JetBot hiện tại.
ID nhảy	Nhiều tag, rung camera hoặc hình ảnh thiếu rõ nét.	Chỉ để một tag trong ảnh, giữ camera ổn định và cải thiện ánh sáng.

Xem thêm

- AprilTag Python package: <https://pypi.org/project/apriltag/>

BÀI 6. Điều hướng đến một tag

Mục	Nội dung
Tài liệu nền	Tiết 29 - AprilTag Navigation phần 1
Mục tiêu	Robot tìm ID 1 và dừng gần 0.30 m.
Sản phẩm	Bảng 3 lần chạy và ảnh/video xe dừng.

Cần những gì

- Vùng trống 1.5 m x 1.5 m.
- Một tag ID 1 dựng thẳng.
- Motor/camera OK; luôn có người quan sát.

Vật tư giáo viên và chuẩn bị trước giờ

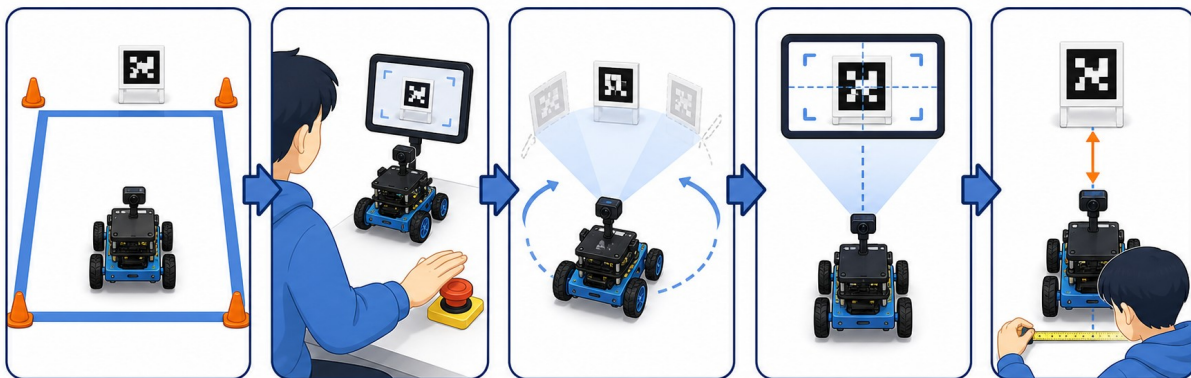
- Khu vực chạy 1.5 m x 1.5 m; băng dán vạch an toàn.
- Tag ID 1 dựng chắc; có người hỗ trợ dừng robot khi cần.

Giải thích dễ hiểu

Navigation gồm scan (xoay tìm đúng ID) và approach (dùng x_m căn hướng, z_m tiến/dừng). Chỉ chạy xe khi Bài 5 đã đọc ID/z_m ổn định.

Quy trình làm

Sơ đồ trực quan: chuẩn bị vùng an toàn → kiểm tra nút dừng → quét tag → căn hướng → tiến chậm và dừng.



25. Khởi tạo camera và Robot.
26. Chạy `scan_for_tag` cho ID 1.
27. Khi thấy ID 1, chạy `approach_tag` mục tiêu 0.30 m.
28. Đo khoảng cách dừng thật.
29. Thử 3 lần, ghi thông số.

30. Cleanup camera và robot.

Mã mẫu

Chạy theo từng cell. Nếu notebook của lớp có tên hàm khác, giữ nguyên ý tưởng và thay bằng tên đúng trong notebook.

```
from jetbot import Robot
import time
robot=Robot(); DESIRED_Z=.30
# tag_state phải được detector cập nhật liên tục: {"id":1,"x_m":...,"z_m":...}
def approach(tag_state):
    while tag_state.get("z_m",99)>DESIRED_Z+.04:
        x=tag_state.get("x_m",0)
        if abs(x)>.10: robot.left(-.04 if x>0 else .04); robot.right(.04 if x>0 else -.04)
        else: robot.left(.15); robot.right(.15)
        time.sleep(.08)
    robot.stop()
```

Tự kiểm tra

- Giải thích scan tìm tag, approach tiến đến tag.
- Robot dừng và không chạy không giám sát.

Lỗi hay gặp

Hiện tượng	Vì sao xảy ra?	Cách xử lý theo thứ tự
Robot không chạy	Motor/cáp chưa sẵn sàng hoặc cell Robot chưa chạy.	Dừng thử nghiệm; kiểm tra motor, cáp và chạy lại cell Robot trong notebook gốc.
Scan timeout	Robot không thấy đúng tag trong vùng quét.	Kiểm tra ID, ánh sáng, góc/độ cao tag; tăng timeout có kiểm soát.
Vượt tag	Tốc độ quá cao, TAG_SIZE hoặc calibration làm ước lượng khoảng cách sai.	Giảm forward speed; kiểm tra TAG_SIZE và calibration trước khi chạy lại.

Xem thêm

- JetBot Robot API: <https://jetbot.org/master/robot.html>

BÀI 7. Tinh chỉnh và đi qua hai tag

Mục	Nội dung
Tài liệu nền	Tiết 30 - AprilTag Navigation phần 2
Mục tiêu	Robot đến ID 1 rồi tiếp tục tìm ID 2; biết chỉnh tham số dựa vào dữ liệu.
Sản phẩm	Bảng tinh chỉnh và ảnh/video robot đến lần lượt hai tag.

Cần những gì

- Bài 6 đã chạy ổn.
- Tag ID 1 và ID 2 cách nhau trên 1 m.
- Thước đo, vùng trống.

Vật tư giáo viên và chuẩn bị trước giờ

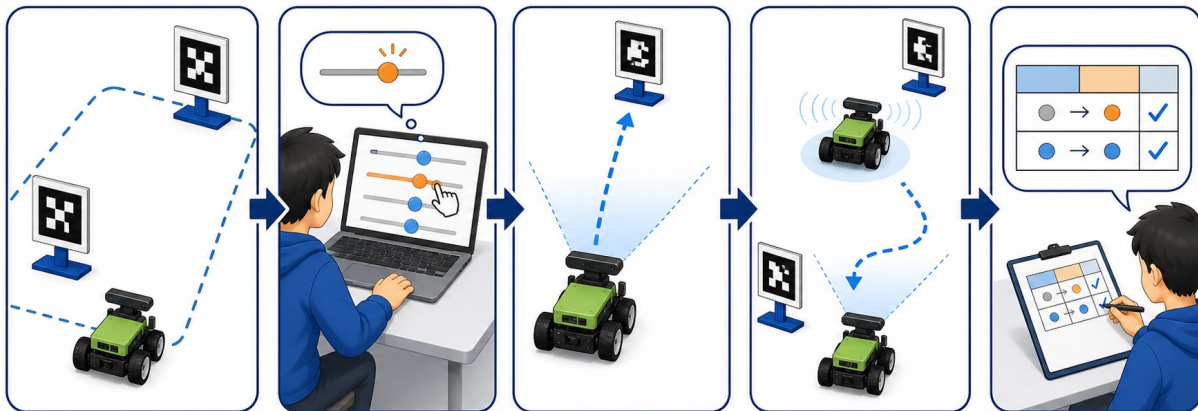
- Hai tag ID 1 và ID 2; khu vực rộng hơn 2 m x 2 m.
- Bảng ghi số liệu tinh chỉnh cho từng nhóm và thước đo.

Giải thích dễ hiểu

forward_speed cao giúp xe nhanh nhưng dễ vượt đích. stop_tolerance nhỏ giúp dừng sát hơn nhưng mất thời gian căn chỉnh. Mỗi lần chỉ đổi một tham số để biết điều gì tạo ra thay đổi.

Quy trình làm

Sơ đồ trực quan: đặt hai tag → chỉ đổi một tham số → đến tag 1 → đến tag 2 → ghi kết quả trước/sau.



31. Chạy tag 1 với thông số ban đầu, ghi khoảng cách thật.
32. Nếu vượt: giảm 0.20 xuống 0.15 rồi 0.12.
33. Nếu cần sát hơn: giảm tolerance 0.04 xuống 0.03/0.02.
34. Chạy scan + approach ID 1.
35. Chờ xe ổn định, rồi scan + approach ID 2.

36. Ghi bảng trước/sau và sơ đồ vị trí tag.

Mã mẫu

Chạy theo từng cell. Nếu notebook của lớp có tên hàm khác, giữ nguyên ý tưởng và thay bằng tên đúng trong notebook.

```
for target in [1,2]:
    found=scan_for_tag(target_tag_id=target,direction="left",turn_speed=.06,timeout=30)
    if not found: break
    ok=approach_tag(tag_state,target_tag_id=target,desired_z_m=.30,forward_speed=.15,turn_speed=.04,center_margin=.10,stop_tolerance=.03,timeout=30)
    if not ok: break
    time.sleep(2)
robot.stop()
```

Tự kiểm tra

- Có tối thiểu 3 lần thử một tag và 1 lần thử hai tag.
- Giải thích tham số nào làm xe dừng tốt hơn.

Lỗi hay gặp

Hiện tượng	Vì sao xảy ra?	Cách xử lý theo thứ tự
Không thấy tag 2	Tag 2 ngoài vùng quét, sai ID hoặc bị che/thiếu sáng.	Đặt tag trong vùng nhìn, kiểm tra ID và tăng timeout có giới hạn.
Xoay giật	Tốc độ quay cao hoặc ngưỡng căn tâm quá nhỏ.	Giảm turn_speed hoặc tăng center_margin; chỉ đổi một tham số mỗi lần.
Dừng 0.28 thay vì 0.30	Stop tolerance và sai số đo tạo ra chênh lệch nhỏ.	Đo lại 3 lần; giảm stop_tolerance nếu cần dừng sát hơn và vẫn đảm bảo an toàn.

Xem thêm

- JetBot project documentation: <https://jetbot.org/master/>

PHỤ LỤC. Bảng nộp sản phẩm cuối module

Sản phẩm	Minh chứng
Khái niệm	Bảng quan sát Bài 1
Camera	Ảnh video live
Calibration	camera_calibration.json + RMS
Verify	verify_undistorted.jpg
Perception	Bảng thước/z_m + ảnh bounding box
Navigation	Ảnh/video + bảng tham số

Checklist kết thúc

- Đã chạy cleanup: camera.stop() và robot.stop().
- Đã sao lưu camera_calibration.json.
- Đã ghi lỗi và tham số cho lần học sau.

PHỤ LỤC VẬN HÀNH TRƯỚC KHI DẠY THẬT

Cảnh báo kỹ thuật: các khối mã ở Bài 6-7 chỉ để giải thích logic. Không copy ghép chúng thành chương trình mới. Navigation phải chạy bằng notebook gốc đã được thử trên đúng JetBot của lớp.

1. Chuẩn bị bắt buộc trước buổi học

Hạng mục	Giáo viên phải xác nhận
JetBot	Bật máy, có IP, SSH được, vào được <code>http://<IP>:8888</code> .
Thư mục gốc	Có thư mục <code>/home/jetbot/apriltags/</code> hoặc đường dẫn thực tế tương đương.
Notebook	Ghi đúng tên 3 notebook: Camera Calibration, AprilTag Perception, AprilTag Navigation. Cần thay bằng tên/tệp thật của lớp.
Thư viện	Chạy <code>pip3 install apriltag</code> nếu notebook chưa chạy; reboot sau cài đặt hoặc khi camera lỗi.
Checkerboard	Dùng đúng một file đã in. Đếm góc trong trên tờ đã in; nếu 11 x 8 ô thì đặt <code>CHECKERBOARD=(10,7)</code> .
Dùng khẩn cấp	Mở sẵn cell <code>robot.stop()</code> ; chuẩn bị người đứng cạnh robot.

2. Chạy đúng notebook, không tự ghép hàm

37. Mở JupyterLab, chọn Kernel > Shut Down All Kernels.
38. Camera test: mở notebook camera gốc; chạy Cell 1 -> Cell 2 -> Cell 3; chỉ đi tiếp khi thấy video live.
39. Calibration: mở notebook Camera Calibration; chạy Cell 1-2; chụp 10-15 góc; Finish; giữ `camera_calibration.json` khi RMS dưới 1.0.
40. Verify: chạy cell video khử méo trực tiếp; so sánh cạnh bàn/laptop trước và sau, không chỉ lưu một ảnh.
41. Perception: mở notebook AprilTag Perception; chạy Cell 1-3; chỉ để một tag trong frame; đo 40/60/80 cm và ghi `z_m`.
42. Navigation: chỉ dùng notebook Navigation đã có `scan_for_tag()` và `approach_tag()`; chạy các cell theo thứ tự notebook. Không chép code rời từ sổ tay.
43. Kết thúc mọi notebook dùng widget/video: chạy `camera_link.unlink()` nếu có, `camera.stop()`, rồi `robot.stop()`.

3. Mẫu cleanup an toàn (dùng khi notebook đã tạo `camera_link`)

```
# Chạy ở cell Cleanup của notebook gốc
try:
    camera_link.unlink()
except NameError:
    pass
try:
    camera.stop()
except NameError:
    pass
try:
    robot.stop()
except NameError:
    pass
print("Đã giải phóng camera và dừng robot.")
```

4. Những chỗ cần kiểm tra trên thiết bị thật

- Tên tệp, đường dẫn notebook, cách tạo Robot và tham số camera tùy thuộc image JetBot đang dùng.

- Không tự đổi `Robot.instance()` thành `Robot()` hoặc ngược lại; dùng đúng API trong notebook gốc đã chạy được.
- Nếu thiếu notebook gốc hoặc file checkerboard chuẩn, không tổ chức phần thực hành; cần sao chép bộ tệp đã xác minh từ giáo viên/phòng lab.

PHỤ LỤC VẬN HÀNH DÀNH CHO GIÁO VIÊN

Bổ sung để triển khai sổ tay trong lớp học mà không cần tự suy đoán mã lệnh

MỤC ĐÍCH: Phụ lục này không thay thế các notebook thực hành. Nó cho giáo viên một kịch bản tổ chức lớp, tiêu chí chuyển bước và cách dừng/xử lý sự cố.

1. Quy tắc dùng sổ tay

Sổ tay gốc có mã minh họa. Chỉ chạy mã trong notebook thực tế đã được người phụ trách kỹ thuật bàn giao. Không copy riêng các đoạn mã có `scan_for_tag()`, `approach_tag()` hoặc `tag_state` để chạy độc lập, vì các hàm/biến này phải có sẵn trong notebook.

Mỗi nhóm chỉ được chuyển sang bài tiếp theo khi có sản phẩm đầu ra của bài trước. Một JetBot/camera chỉ nên được một notebook sử dụng tại một thời điểm.

KHÔNG TỰ ĐOÁN: Nếu thiếu notebook, không rõ tên file, không thấy file `camera_calibration.json`, hoặc checkerboard không đúng mẫu: dừng ở bước đó và báo người phụ trách kỹ thuật. Không thay thông số bằng phỏng đoán.

2. Phiếu bàn giao kỹ thuật – phải điền trước khi dạy

Mục cần chốt	Thông tin phải điền	Đã kiểm tra
IP JetBot	JetBot 1: _____ JetBot 2: _____	☐
Thư mục notebook	Ví dụ: <code>/home/jetbot/apriltags/</code> Đường dẫn thực tế: _____	☐
Notebook camera	Tên file: _____	☐
Notebook calibration	Tên file: _____	☐
Notebook Perception	Tên file: _____	☐
Notebook Navigation	Tên file: _____	☐
Checkerboard chuẩn	Tên file/mẫu: _____ số ô: _____ góc trong code: _____	☐
Kích thước cạnh tag	Đo thực tế: _____ cm → <code>TAG_SIZE =</code> _____ m	☐
Cell dừng khẩn cấp	Tên cell/lệnh do kỹ thuật viên xác nhận: _____	☐

3. Checklist giáo viên trước giờ học

- ☐ JetBot đã sạc pin; bánh xe, dây động cơ và camera CSI được gắn chắc.
- ☐ Máy tính và JetBot cùng Wi-Fi; mở được JupyterLab tại `http://<IP>:8888`.
- ☐ Đã thử Shut Down All Kernels, mở notebook đúng tên và thấy camera live trước khi học sinh vào lớp.
- ☐ Đã có tag family tag36h11, ID cần dùng; tag in rõ, dán phẳng lên bìa cứng, dựng đứng được.
- ☐ Đã có checkerboard đúng mẫu. Số góc trong code phải khớp bằng thật, không chỉ dựa vào số ô.
- ☐ Có thước đo, phiếu ghi kết quả, vùng chạy trống, băng dán ranh giới và người quan sát robot.
- ☐ Đã sao lưu `camera_calibration.json` của từng JetBot nếu tiết học dùng calibration cũ.

4. Quy trình an toàn và dừng khẩn cấp

TRƯỚC KHI ROBOT CHẠY: Dọn vùng ít nhất 1,5 m × 1,5 m; đặt tag chắc chắn; không để người đứng trước đường chạy; chỉ chạy tốc độ thấp trong lần thử đầu.

Tình huống	Hành động ngay	Sau đó kiểm tra
Robot chạy sai hướng hoặc tiến quá nhanh	Bấm Interrupt Kernel; chạy cell/lệnh robot.stop() đã bàn giao; nếu vẫn không dừng, tắt nguồn JetBot.	forward_speed, TAG_SIZE, camera_calibration.json, đường chạy.
Camera đứng hình hoặc Device busy	Shut Down All Kernels; chạy lại từ cell đầu. Nếu chưa được, reboot JetBot.	Có notebook khác đang dùng camera; cleanup đã chạy chưa.
Không thấy tag	Dừng robot; không cho robot quét tiếp vô thời hạn.	Đúng tag36h11/ID, ánh sáng, tag phẳng, khoảng cách và toàn bộ tag trong khung hình.

5. Kịch bản triển khai 7 bài

Bài	GV/HS thực hiện	Đạt khi	Lưu ý
Bài 1 – Làm quen tag	Phát tag ID 0–3; HS quan sát, xoay, đưa xa/gần và che một phần. Không cần JetBot.	HS mô tả được ID, 4 góc, tâm; có bảng dự đoán/kết quả.	Không yêu cầu tag phải detect khi bị che; HS ghi kết quả thực tế.
Bài 2 – Camera	Mở notebook Camera đã bàn giao. Shut Down All Kernels → chạy từng cell theo thứ tự.	Video live ổn định ít nhất 2 phút; có ảnh chụp màn hình.	Thiếu notebook/cài thư viện: GV chuyển sang demo, không yêu cầu HS copy mã mẫu.
Bài 3 – Calibration	Dùng đúng checkerboard; chụp 10–15 frame ở giữa, mép, gần, xa và nghiêng.	Lưu camera_calibration.json; ghi RMS. Ưu tiên RMS < 1.0; > 2.0 làm lại.	Mỗi camera cần file riêng; không lấy file JetBot khác dùng thay.
Bài 4 – Verify	Mở notebook Verify; quan sát video gốc và video khử méo, nhất là ở rìa ảnh; chạy cleanup.	Ghi RMS, fx/fy/cx/cy; đường thẳng ở mép ít cong hơn.	Nếu không có JSON hoặc RMS > 2.0: quay lại Bài 3.
Bài 5 – Perception	Mở notebook Perception; dùng một tag/lần; đo 40, 60, 80 cm rồi đổi chiều z_m.	Đọc đúng ID, z_m, x_m; có ảnh bounding box và bảng số liệu.	TAG_SIZE phải là cạnh thật của tag tính bằng mét.
Bài 6 – Một tag	Chỉ dùng notebook Navigation hoàn chỉnh. Cho robot cách tag > 0,5 m; chạy scan rồi approach với người quan sát.	Robot tìm ID 1 và dừng gần 0,30 m; đo 3 lần.	Lần thử đầu dùng tốc độ thấp. Sai nhiều: dừng và kiểm tra calibration/TAG_SIZE trước.
Bài 7 – Hai tag	Sau khi Bài 6 ổn, đặt ID 1 và 2 cách nhau > 1 m. Chạy hai lượt scan + approach trong notebook.	Có dữ liệu trước/sau khi chỉnh tốc độ và tolerance; đến ID 1 rồi ID 2.	Mỗi lần chỉ thay một tham số để biết nguyên nhân thay đổi.

6. Tiêu chí kết quả và sản phẩm phải nộp

Nội dung	Mình chứng tối thiểu	Tiêu chí đạt
Camera	Ảnh màn hình video live	Video không đứng trong 2 phút.
Calibration	camera_calibration.json + RMS	RMS < 1.0 tốt; 1.0–2.0 chấp nhận; > 2.0 làm lại.
Verify	Ảnh/video khử méo + ghi chú	Nêu được sự khác biệt ở rìa ảnh.
Perception	Bảng đo 40/60/80 cm + ảnh bounding box	ID đúng; z_m thay đổi đúng chiều khi tag gần/xa.
Navigation 1 tag	3 lần đo khoảng cách dừng	Robot dừng gần 0,30 m trong sai số do GV quy định.
Navigation 2 tag	Bảng tham số và video/ảnh	Hoàn thành ID 1 rồi ID 2 an toàn.

7. Biểu mẫu ghi kết quả

Phiếu A – Đo khoảng cách AprilTag

Khoảng cách thật (cm)	z_m hiển thị (m)	Sai số (cm)	ID / Hamming / Margin	Ghi chú
40				
60				
80				

Phiếu B – Ba lần chạy đến tag ID 1

Lần	forward_speed	stop_tolerance	Khoảng cách dừng thật	Kết quả / lỗi
1				
2				
3				

Phiếu C – Bảng tinh chỉnh

Chỉ thay đổi	Giá trị trước	Giá trị sau	Robot thay đổi thế nào?	Kết luận
forward_speed / stop_tolerance / khác				

8. Checklist kết thúc tiết

- ☐ Đã chạy cell cleanup: dừng robot, dừng camera và ngắt liên kết widget nếu notebook có liên kết đó.
- ☐ Đã sao lưu camera_calibration.json (nếu có tạo/cập nhật).
- ☐ Đã thu phiếu kết quả hoặc ảnh/video minh chứng của từng nhóm.
- ☐ Đã ghi lỗi kỹ thuật, JetBot gặp lỗi và việc cần chuẩn bị cho tiết sau.
- ☐ Đã tắt nguồn/sạc pin JetBot theo quy định của phòng học.